CISC 7403 Project I (16%): Microservice Workload with Container Isolation

**Course Feedback System**

# 1 Background

Cloud computing changes the way applications are packaged, deployed, and managed. Modern applications are often divided into multiple services so that different functions can be developed and operated independently. Container technologies such as Docker provide a lightweight way to package these services and isolate their runtime environments. Docker Compose further allows multiple containers to be deployed and connected as one application.

# 2 Objective

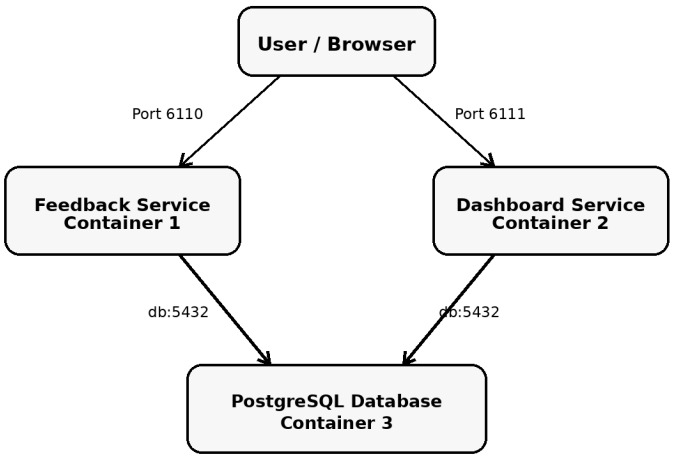
By the end of this project, you should be able to design, develop, containerize, and deploy a simple multi-service workload using Docker. You should also understand basic container networking and application logging in a containerized environment.

# 3 Requirements

## 3.1 Cloud Computing and Container Setup

- Install Docker on the virtual machine provided for this course.
- Build a Course Feedback System based on the architecture and requirements below.

You need to develop a system that includes 3 services running in 3 separate containers: feedback, dashboard, and database.



1. Feedback service

Required external port: 6110.

This service provides a web page for students to submit feedback about the course. The page should contain the following inputs:

- Difficulty rating: an integer from 1 (very easy) to 5 (very difficult).
- Favorite topic: choose one from Virtual Machine, Docker, Kubernetes, or Serverless.

(a) When the user submits the form, this service should insert one feedback record into the database.

(b) After a successful submission, the frontend should clearly indicate that the feedback has been submitted successfully.

(c) The service should log each successful submission, including the difficulty rating and selected topic. These logs must be visible using the command “docker logs” or “docker compose logs”.

Example frontend (style is for reference only):

| **Course Feedback** |
|---|
| How difficult is this course? 1 2 3 4 5 |
| Favorite topic: Virtual Machine / Docker / Kubernetes / Serverless |
| [ Submit Feedback ] |
| Feedback submitted successfully! |

2. Dashboard service

Required external port: 6111.

This service provides a read-only web page that summarizes the feedback stored in the database. The page should display:

- the total number of feedback records;
- the average difficulty rating;
- for each topic, the number of selections.

The dashboard must retrieve its data from the PostgreSQL database. Automatic refresh is not required; refreshing the browser page is sufficient.

3. Database service

Use PostgreSQL as a separate database container. The database does not need to expose a port to the host machine. The two application services should access it through the Docker Compose internal network.

Create one simple table for storing feedback records. A suggested schema is shown below:

| **Column** | **Type** | **Example** | **Description** |
|---|---|---|---|
| id | SERIAL / INTEGER | 1 | Unique record ID |

| | | | |
|---|---|---|---|
| difficulty | INTEGER | 4 | Difficulty rating from 1 to 5 |
| topic | VARCHAR | Docker | Selected course topic |

- Create a Dockerfile for the feedback service and another Dockerfile for the dashboard service.
- Use an official PostgreSQL image for the database service.
- Create a docker-compose.yml file to deploy the whole system.
- The application containers should connect to the database using the Compose service name (for example, db), not localhost.
- Only ports 6110 and 6111 need to be exposed to the host. The PostgreSQL port should remain internal unless you have a specific debugging need.

You do not need to follow the frontend style shown in this document. A simple HTML interface is sufficient. No charts, authentication, automatic refresh, or other extra features are required.

## 3.2 Development of Microservices

Create 3 containers to deploy the three services using Docker Compose. The final system should be startable with a single Docker Compose command. The feedback and dashboard containers should communicate with the database container through the Docker network created by Compose.

Your submission should work after running:

**docker compose up --build**

## 3.3 Basic Container Requirements

Your deployment should demonstrate the following basic container behaviors:

- Networking: the application containers reach PostgreSQL through the Compose service name.
- Logging: feedback submission logs can be inspected from the host using Docker logging commands.

## 3.4 Scoring

| Item | Weight |
|---|---|
| Feedback service functionality | 20% |
| Dashboard service functionality | 15% |
| Database service | 10% |
| Docker containerization, Compose deployment, networking, and logs | 35% |
| Report | 20% |

## 3.5 Deliverables

Write a brief report (maximum 3 pages, excluding the cover page and source code). The report should include:

(1) A brief description of the system architecture and the functionality of the three services.
(2) A brief description of how the services are containerized and deployed using Docker Compose.
(3) Evidence that the system works, including screenshots of the feedback page, dashboard page, running containers, and application logs.
(4) Any challenges you faced and a member contribution statement.
(5) The role declaration page must be filled, signed and submitted by each student individually.

*__In addition to the report, the complete source code, Dockerfiles, and docker-compose.yml must be submitted as attachments for grading.__

# 4 Reference

- Python tutorial: https://www.w3schools.com/python/ or https://www.runoob.com/python/python-tutorial.html (Chinese version)
- docker project: https://github.com/zhenqiang-sun/docker-python-demo/tree/master
- docker for beginners: https://docker-curriculum.com

# 5 Basic Tutorials

- Docker install:https://docs.docker.com/engine/install/ubuntu/
- Docker build:https://docs.docker.com/build/building/packaging/

  You need to build a Dockerfile, and then create a container image based on the Dockerfile.

- Docker Compose: https://docs.docker.com/compose/
- Communication: different dockers can build connections with each other via HTTP: https://realpython.com/python-http-server/
- PostgreSQL: https://www.w3schools.com/postgresql/ or https://www.runoob.com/postgresql/postgresql-tutorial.html (Chinese version)